

Vigía: Sistema de detección temprana de humo y fuego mediante visión artificial

TRABAJO FIN DE MÁSTER



Máster en Big Data, Data Science e Inteligencia Artificial

Alumno: Elías Ruiz Fernández

Septiembre de 2026

Índice

1. Introducción y objetivos	1
1.1. Motivación	1
1.2. Objetivos	1
2. Estado del arte	1
2.1. Detección de objetos en visión por computador	1
2.2. La familia YOLO	2
2.3. Aplicación al problema de humo y fuego	3
3. Datos	3
3.1. Conjunto de datos	3
3.2. Limpieza y revisión	3
3.3. Distribución de las particiones	4
3.4. Ejemplos del dataset	5
4. Metodología y modelos desarrollados	6
4.1. Comparación inicial de modelos	6
4.2. Estudio de resolución de entrada	7
4.3. Comparación a 768 píxeles y optimización	8
4.4. Ajuste de hiperparámetros de YOLO26s	9
4.5. Selección final en validación	10
5. Evaluación y resultados	12
5.1. Métricas en test	12
5.2. Recall por tamaño de objeto	13
5.3. Análisis de errores	14
5.4. Ejemplos de inferencia	14
6. Interpretabilidad	15
7. Despliegue y productivización	18
7.1. Raspberry Pi 5	18
7.2. Aplicación de detección	19
8. Conclusiones y trabajo futuro	19
9. Bibliografía y recursos	21
A. Acceso al repositorio del proyecto	22
B. Descripción de las métricas de evaluación	23
B.1. Confianza, IoU y emparejamiento de cajas	23
B.2. Verdaderos positivos, falsos positivos y falsos negativos	23
B.3. Precisión, recall y F1	23
B.4. AP, mAP50 y mAP50-95	24
B.5. Métricas operativas propias del proyecto	24
C. Arquitectura de YOLO26	25
C.1. Entrada y extracción de características	25

C.2. Fusión multiescala	26
C.3. Cabeza de detección	26
C.4. Escalas de modelo y despliegue	26

1. Introducción y objetivos

1.1. Motivación

Los incendios forestales han dejado de ser un riesgo estacional puntual para convertirse en una emergencia recurrente con impacto directo sobre personas, infraestructuras y ecosistemas. En España, el verano de 2025 fue especialmente ilustrativo: según el avance oficial del Ministerio para la Transición Ecológica y el Reto Demográfico, durante ese año se registraron 8.199 siniestros y ardieron 354.746,67 hectáreas de superficie forestal, un 235,89 % por encima de la media de la década 2015–2024 [1].

El comienzo de la campaña de 2026 confirma que no se trata de un episodio aislado. El avance provisional más reciente disponible en la web del MITECO, con datos acumulados hasta el 30 de agosto de 2026, recoge 7.431 siniestros, 265.635,99 hectáreas forestales afectadas y 44 grandes incendios forestales, frente a una media decenal para esa fecha de 97.879 hectáreas y 23 grandes incendios [2].

En este contexto, reducir el tiempo entre la aparición de una señal visible y el aviso a los cuerpos de seguridad se convierte en una necesidad. Este proyecto explora la implementación de un detector visual de humo y fuego autónomo que pueda integrarse en sistemas de vigilancia existentes como cámaras o drones.

1.2. Objetivos

El objetivo principal de este trabajo es diseñar, evaluar y desplegar un sistema de visión por computador capaz de detectar humo y fuego en imágenes, vídeos y cámara en directo, con un coste computacional compatible con dispositivos de recursos limitados. Además, el sistema debe confirmar la persistencia de las detecciones antes de generar una alerta y comunicar los incidentes mediante una interfaz accesible para el usuario. Para alcanzarlo, se plantean los siguientes objetivos:

- Preparar y documentar un conjunto de datos de imágenes de humo y fuego, con particiones trazables para entrenamiento, validación y test.
- Entrenar y comparar modelos de detección visual, evaluando distintas arquitecturas, resoluciones de entrada y configuraciones de entrenamiento.
- Analizar la interpretabilidad del modelo mediante métricas y ejemplos visuales que permitan comprobar qué regiones de la imagen sostienen las predicciones.
- Desarrollar una aplicación que permita analizar imágenes, vídeos y cámara en directo, visualizar los resultados y gestionar alertas de forma cómoda para el usuario.
- Optimizar y medir el funcionamiento del sistema en un dispositivo de bajo coste, como la Raspberry Pi 5.

2. Estado del arte

2.1. Detección de objetos en visión por computador

La detección de objetos es una de las tareas centrales de la visión por computador. A diferencia de la clasificación de imágenes, cuyo objetivo es asignar una o varias etiquetas

globales a una imagen completa, la detección debe localizar cada instancia relevante mediante una caja delimitadora y asociarle una clase y una puntuación de confianza. En el caso de este trabajo, esto implica no solo decidir si existe humo o fuego, sino también indicar en qué región de la imagen aparece la señal.

Los detectores modernos suelen estar basados en redes neuronales profundas y se entrenan con imágenes anotadas mediante cajas. Durante la inferencia, el modelo produce un conjunto de predicciones formado por coordenadas, clase y confianza. Posteriormente, estas predicciones se filtran mediante umbrales y, en muchos modelos, mediante técnicas de supresión de cajas redundantes. La calidad del detector depende de su capacidad para localizar correctamente los objetos, distinguir clases visualmente próximas y mantener un equilibrio adecuado entre sensibilidad y falsas alarmas.

Desde el punto de vista arquitectónico, los detectores se suelen agrupar en dos grandes familias: detectores de dos etapas y detectores de una etapa. Los detectores de dos etapas (*Two-Stage Detectors*) generan primero regiones candidatas y después clasifican y refinan dichas regiones. Faster R-CNN es uno de los referentes de esta familia [5]. Esta aproximación ha sido históricamente muy competitiva en precisión, pero su coste computacional y su latencia la hacen menos atractiva cuando se busca procesamiento en tiempo real o despliegue en dispositivos con recursos limitados.

Los detectores de una etapa (*One-Stage Detectors*), por el contrario, formulan la detección como una predicción directa sobre la imagen o sobre mapas de características. SSD (*Single Shot MultiBox Detector*) es un ejemplo representativo de este grupo [6]. Esta familia reduce la latencia y simplifica la integración en aplicaciones donde el detector debe ejecutarse de forma continua.

2.2. La familia YOLO

YOLO (*You Only Look Once*) surge precisamente dentro de esta filosofía de detección en una sola etapa. Su idea original consiste en formular la detección como un problema de regresión: una única red procesa la imagen completa y predice directamente cajas delimitadoras y probabilidades de clase en una sola evaluación [7]. Esta formulación unificada hizo que YOLO destacara desde sus primeras versiones por su velocidad de inferencia, una propiedad especialmente relevante para vídeo, cámara en directo y sistemas embebidos.

A lo largo de sus distintas versiones, YOLO ha evolucionado desde arquitecturas convolucionales relativamente simples hasta modelos más modulares y optimizados. En términos generales, las implementaciones modernas de YOLO mantienen una estructura dividida en tres bloques: un *backbone*, encargado de extraer características visuales a distintas resoluciones; un *neck*, que fusiona información de varias escalas para combinar contexto semántico y detalle espacial; y una *head*, que transforma esas características fusionadas en cajas, clases y puntuaciones de confianza [8]. La arquitectura de YOLO26 se describe con mayor detalle en el Anexo C.

Esta división resulta especialmente útil en problemas como la detección de humo y fuego. El humo puede ocupar regiones amplias y difusas, con bordes poco definidos, mientras que las llamas pueden aparecer como objetos pequeños, saturados y de geometría irregular. Por ello, el detector debe aprovechar tanto información local, necesaria para delimitar regiones pequeñas, como contexto global, necesario para interpretar patrones de humo o descartar elementos visualmente parecidos.

2.3. Aplicación al problema de humo y fuego

La detección visual de humo y fuego presenta dificultades específicas frente a otros problemas de detección de objetos. Por un lado, las clases de interés no siempre tienen una forma rígida: el humo cambia con el viento, la iluminación y el fondo, y el fuego puede variar en color, intensidad y tamaño. Por otro lado, existen falsos positivos plausibles, como nubes, niebla, reflejos, luces intensas o superficies rojizas.

Estas características hacen que el problema no consista únicamente en encontrar objetos bien delimitados. En muchas escenas, la señal de riesgo aparece de forma gradual, parcial o ambigua: una pequeña llama puede confundirse con una luz intensa, y una zona de humo puede parecerse a una nube o a niebla.

En este contexto, YOLO resulta adecuado porque permite analizar imágenes y secuencias de vídeo con baja latencia, localizar las regiones sospechosas y mantener un coste computacional compatible con dispositivos de recursos limitados. Su utilidad para este trabajo no se limita a detectar humo o fuego en una imagen individual, sino a servir como base de una solución operativa que pueda funcionar sobre imagen, vídeo o cámara en directo.

3. Datos

3.1. Conjunto de datos

El proyecto utiliza el conjunto de datos *D-Fire*, una colección pública orientada a detección visual de humo y fuego [3]. El repositorio oficial indica que la base de imágenes se distribuye bajo la licencia Creative Commons Zero v1.0 Universal (CC0 1.0) y recomienda citar el trabajo de Venâncio, Lisboa y Barbosa cuando se utiliza dicha base de imágenes [4]. Cada imagen puede contener ninguna señal de incendio, solo humo, solo fuego o ambas clases a la vez. Las anotaciones proporcionadas son coordenadas de cajas delimitadoras, lo que permite entrenar un detector que no solo clasifique imágenes, sino también dónde aparecen los objetos dentro de las mismas.

La distribución utilizada contiene 17.221 imágenes de entrenamiento y 4.306 imágenes de test. Para disponer de una partición de validación independiente, se reservó el 10 % del conjunto de entrenamiento mediante una división estratificada por tipo de imagen y con semilla fija, de forma que la validación mantuviera una distribución de categorías similar a la del entrenamiento. El resultado final fue: 15.500 imágenes para entrenamiento, 1.721 para validación y 4.306 para test.

3.2. Limpieza y revisión

Antes del entrenamiento se creó una versión revisada y particionada del dataset. Esta preparación queda documentada en el notebook 01 (Anexo A), que, a partir de las imágenes y anotaciones originales, revisa las 21.527 imágenes, valida que las rutas y las etiquetas sean legibles, comprueba el mapeo de clases y genera un manifiesto único con la partición asignada a cada imagen. Además, las imágenes fueron revisadas visualmente para detectar posibles errores de anotación o inconsistencias.

La revisión permitió detectar y corregir tanto anotaciones que cruzaban los límites de la

imagen como anotaciones con dimensiones inválidas. Todas estas correcciones quedaron registradas en ficheros de auditoría, con el fin de mantener la trazabilidad del proceso.

Por último, la preparación incorpora una comprobación de duplicados exactos entre particiones. Este control no garantiza la ausencia de escenas visualmente parecidas, pero evita el caso más directo de fuga de información: que una misma imagen aparezca en más de una partición.

3.3. Distribución de las particiones

La Figura 1 resume el número de imágenes por tipo en entrenamiento, validación y test. La clase más frecuente no corresponde a fuego aislado, sino a imágenes sin anotaciones y a imágenes con humo. Esto es relevante porque el sistema debe aprender tanto a detectar señales de incendio como a no activar alertas ante escenas visualmente parecidas pero sin anotación de riesgo.

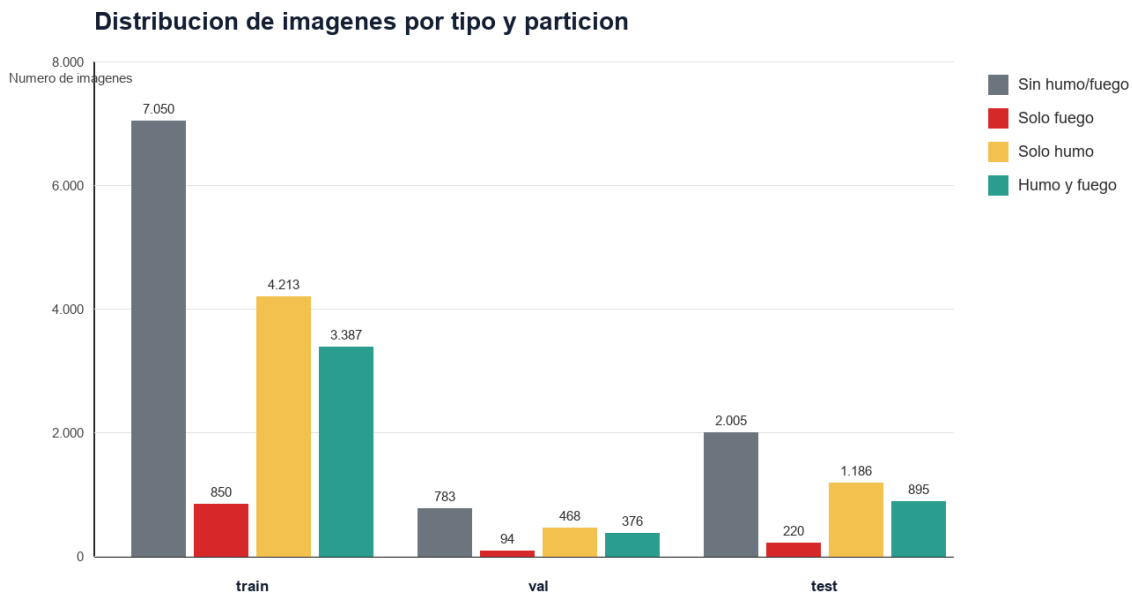


Figura 1: Distribución de imágenes por tipo y partición. La validación procede del 10 % del entrenamiento original.

La Figura 2 muestra el número de cajas anotadas de humo y fuego. Una imagen puede contener más de una caja, por lo que estos valores no coinciden necesariamente con el número de imágenes de cada tipo. Se observa como la proporción de cajas de cada clase es similar en todas las particiones.

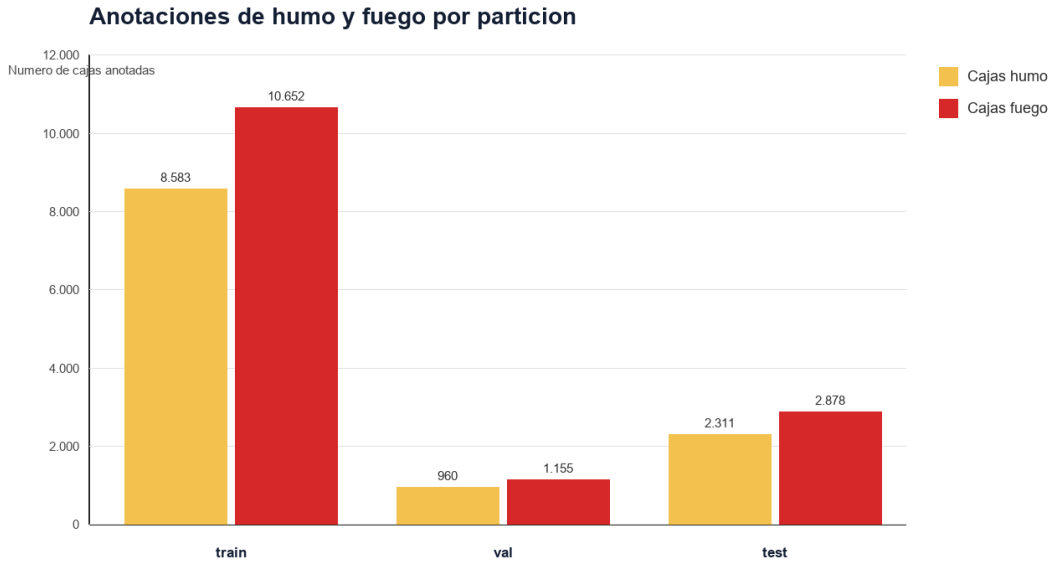


Figura 2: Número de cajas anotadas de humo y fuego en cada partición.

3.4. Ejemplos del dataset

La Figura 3 muestra ejemplos representativos de las categorías utilizadas durante la preparación. Se incluyen imágenes sin anotaciones, con humo, con fuego y con ambas clases. La variedad visual ilustra por qué el problema es complejo: el humo puede ser difuso y ocupar una región amplia, mientras que el fuego puede aparecer como una señal pequeña, brillante o parcialmente oculta.



Figura 3: Ejemplos de imágenes del dataset preparado: fondo sin anotaciones, humo, fuego y humo con fuego.

4. Metodología y modelos desarrollados

4.1. Comparación inicial de modelos

El desarrollo se planteó como un proceso incremental. Una vez preparado y revisado el dataset, se entrenaron varias familias y tamaños de detectores YOLO manteniendo constantes las particiones, la semilla y el protocolo de evaluación. Se escogió YOLOv8 como línea base porque es una familia ampliamente utilizada, estable y bien documentada en detección de objetos. YOLO26 se incluyó como alternativa más reciente y eficiente. En ambos casos se limitaron los ensayos a las variantes nano y small, ya que el objetivo final del proyecto era construir un sistema desplegable en un entorno con recursos acotados, no maximizar la métrica con modelos de gran tamaño.

Los entrenamientos se realizan con ayuda del notebook 02. El proceso de evaluación de cada modelo se lleva a cabo con el notebook 03, exclusivamente sobre el conjunto de validación; el conjunto de test permanece intacto hasta que el modelo y los umbrales quedan congelados en la etapa de selección final. Las rutas completas de estos notebooks se recogen en el Anexo A.

Todos los modelos de esta primera comparación comparten la misma configuración base de hiperparámetros: 100 épocas, *batch size* 16 y tamaño de imagen 640×640, optimizador AdamW con tasa de aprendizaje inicial 0,001 y programación por coseno, aumento de datos por mosaico (desactivado en las últimas 10 épocas) y parada temprana con paciencia de 20 épocas sin mejora. Todos los entrenamientos se ejecutaron en una GPU NVIDIA RTX 5070.

Tabla 1: Comparación inicial sobre validación. Las métricas operativas se obtienen con confianza predeterminada de 0,25 e IoU de emparejamiento 0,50.

Modelo	mAP50	mAP50-95	Precisión micro	Recall micro	F1 micro	Neg. con alarma	Parámetros
YOLOv8n	78,05 %	44,63 %	68,91 %	75,56 %	72,08 %	19/783	3,01 M
YOLOv8s	79,44 %	46,42 %	70,77 %	77,73 %	74,09 %	18/783	11,13 M
YOLO26n	76,89 %	44,87 %	72,12 %	72,06 %	72,09 %	6/783	2,38 M
YOLO26s	79,28 %	46,11 %	73,16 %	74,75 %	73,95 %	7/783	9,47 M

Tabla 2: Desglose por clase del mAP50-95 en validación. El fuego se comporta de forma sistemáticamente más difícil que el humo en los cuatro modelos.

Modelo	mAP50-95 humo	mAP50-95 fuego
YOLOv8n	51,73 %	37,53 %
YOLOv8s	53,65 %	39,19 %
YOLO26n	52,57 %	37,18 %
YOLO26s	52,90 %	39,31 %

YOLOv8s obtuvo el mayor mAP50-95, pero la diferencia con YOLO26s fue muy reducida: 0,31 puntos porcentuales. YOLO26s ofreció un F1 prácticamente equivalente en el punto común de confianza 0,25, con menor tamaño de pesos y menos alarmas en imágenes negativas. Por ello se tomó como base para las siguientes etapas, dejando YOLOv8s como segunda opción YOLO26n como alternativa más ligera. El desglose por clase de la Tabla 2 confirma que el fuego es la clase más difícil de detectar en los cuatro modelos, con un mAP50-95 entre

37,18 % y 39,31 % frente a 51,73 %-53,65 % en humo; esta asimetría motivó más adelante la separación de los umbrales de decisión por clase. No se incluye el tiempo de inferencia en esta comparación inicial porque se midió en el equipo de entrenamiento (GPU NVIDIA RTX 5070) y no es representativo del dispositivo de despliegue final; el rendimiento en el hardware objetivo se trata en la sección de despliegue.

4.2. Estudio de resolución de entrada

En esta segunda etapa se analizó el efecto de la resolución de entrada sobre la detección de humo y fuego, con especial atención a este último ya que es el que peores métricas obtuvo. El estudio se realizó únicamente con YOLOv8s, en lugar de repetirlo también con YOLO26s, para acotar el coste computacional a una sola arquitectura, ya que los tiempos de entrenamiento a mayor resolución son considerablemente más largos. Se entrenaron de forma independiente tres modelos, a 640, 768 y 1024 píxeles, usando los mismos hiperparámetros que en el apartado anterior, y cada uno se evaluó a su vez a las tres resoluciones de inferencia. En cada configuración, el umbral de decisión se eligió para maximizar el recall macro de humo y fuego sin superar un máximo del 2 % de imágenes negativas con alarma. Este análisis se recoge en el notebook 06.

Este criterio (maximizar el recall fijando un límite de falsas alarmas, en lugar de optimizar directamente el F1 u otra métrica simétrica) se emplea de forma sistemática en todo el proyecto porque las dos clases de error no tienen el mismo coste en un sistema de aviso temprano: un falso negativo significa no detectar un incendio real, mientras que un falso positivo es una alarma molesta pero corregible. El objetivo es, por tanto, aumentar la sensibilidad tanto como sea posible sin degradar de forma inaceptable el resto de métricas, especialmente la precisión y el F1.

Maximizar el recall sin ninguna restricción llevaría trivialmente a marcar cualquier imagen como positiva, lo que en la práctica inutilizaría el sistema por exceso de avisos y erosionaría la confianza del usuario. Por ello, primero se fija un límite operativo de falsas alarmas considerado asumible y, dentro de esa región factible, se selecciona el umbral que maximiza la sensibilidad. El límite concreto no es el mismo en todas las etapas del proyecto: en esta comparación se usa un 2 % porque el objetivo es evaluar resoluciones con cierto margen operativo; más adelante, en la selección final (Sección 4.5), se reduce hasta aproximadamente el 1 %, al tratarse ya de la configuración candidata para despliegue.

Tabla 3: Efecto de la resolución sobre validación, para las nueve combinaciones de resolución de entrenamiento y de inferencia.

Configuración	Umbral	Recall humo	Recall fuego	Precisión micro	F1 micro	Neg. con alarma
640 → 640	0,30	82,08 %	70,48 %	74,65 %	75,19 %	13/783
640 → 768	0,30	81,15 %	71,52 %	73,83 %	74,84 %	11/783
640 → 1024	0,40	70,73 %	63,38 %	75,49 %	70,83 %	12/783
768 → 640	0,30	80,94 %	73,16 %	74,64 %	75,65 %	13/783
768 → 768	0,30	81,04 %	71,17 %	73,46 %	74,54 %	15/783
768 → 1024	0,35	79,90 %	68,74 %	69,66 %	71,67 %	15/783
1024 → 640	0,20	83,44 %	74,89 %	66,56 %	72,15 %	14/783
1024 → 768	0,25	82,40 %	72,73 %	71,10 %	73,99 %	14/783
1024 → 1024	0,25	83,54 %	73,51 %	70,71 %	74,20 %	14/783

La Tabla 3 incluye las nueve combinaciones de resolución de entrenamiento y de inferencia. Entrenar a 768 píxeles y evaluar a 640 obtiene el mejor F1 micro de las nueve combinaciones (75,65 %) y un recall de fuego del 73,16 %, por encima del 70,48 % de la configuración base a 640, con una precisión prácticamente idéntica (74,64 % frente a 74,65 %) y sin pagar el coste de inferencia de evaluar también a 768 píxeles. Entrenar a 1024 y evaluar a 640 alcanza el mayor recall de fuego de toda la rejilla (74,89 %), pero a costa de una precisión notablemente menor (66,56 %) y de un coste de entrenamiento mucho mayor, por lo que no se considera un mejor compromiso. Cualquier inferencia a 1024 píxeles también se descarta por el alto coste de cómputo en comparación al resto de resoluciones (Tabla 4). Por este motivo, la configuración “entrenado a 768, evaluado a 640” se toma como referencia para las siguientes etapas del proyecto para el modelo YOLOv8.

Tabla 4: Coste de inferencia en validación según la resolución de evaluación para el dispositivo de entrenamiento (NVIDIA GeForce RTX 5070).

Resolución de evaluación	Inferencia (ms/imagen)
640	1,70
768	2,18
1024	4,14

4.3. Comparación a 768 píxeles y optimización

Después del análisis del apartado anterior, YOLO26s se entrenó a 768 píxeles omitiendo la configuración a 1024: el estudio con YOLOv8s ya había mostrado que entrenar a 1024 no resultaba rentable, con un coste de entrenamiento 2,5 veces mayor y de inferencia más del doble frente a 768, sin una mejora de recall que lo compensase. El entrenamiento de YOLO26s a 768 utilizó los mismos hiperparámetros e idéntico dataset que la comparación inicial.

A diferencia de YOLOv8s, donde evaluar a una resolución menor que la de entrenamiento (768 → 640) mejoraba el punto operativo, en YOLO26s ocurre lo contrario. Ambas arquitecturas, entrenadas a 768, se evaluaron con inferencia a 640 y a 768 bajo el mismo criterio de selección (máximo recall macro con un límite del 2 % de imágenes negativas con alarma), como recoge la Tabla 5. La Figura 4 muestra estas mismas tendencias a lo largo de todo el barrido de umbrales, no solo en el punto operativo.

Tabla 5: Comparación de arquitecturas entrenadas a 768 píxeles, evaluadas a 640 y a 768.

Configuración	Umbral	Recall humo	Recall fuego	Recall macro	F1 micro	mAP50-95	Neg. con alarma
YOLOv8s 768 → 640	0,26	81,88 %	74,89 %	78,38 %	74,76 %	46,52 %	15/783
YOLOv8s 768 → 768	0,28	82,19 %	72,12 %	77,15 %	74,15 %	46,57 %	15/783
YOLO26s 768 → 640	0,18	83,65 %	76,19 %	79,92 %	72,57 %	46,40 %	14/783
YOLO26s 768 → 768	0,16	85,42 %	78,53 %	81,97 %	72,37 %	46,68 %	12/783

Se observa cómo en YOLOv8s, evaluar a 640 en lugar de a 768 mejora el recall macro (78,38 % frente a 77,15 %), lo que justifica emplear la configuración “768 → 640” como referencia para esta arquitectura. En YOLO26s ocurre justo lo contrario: evaluar a su resolución nativa de entrenamiento (768) obtiene mayor recall macro (81,97 % frente a 79,92 %) y menos

falsas alarmas (12/783 frente a 14/783), aunque el F1 micro sea prácticamente igual en ambas resoluciones.

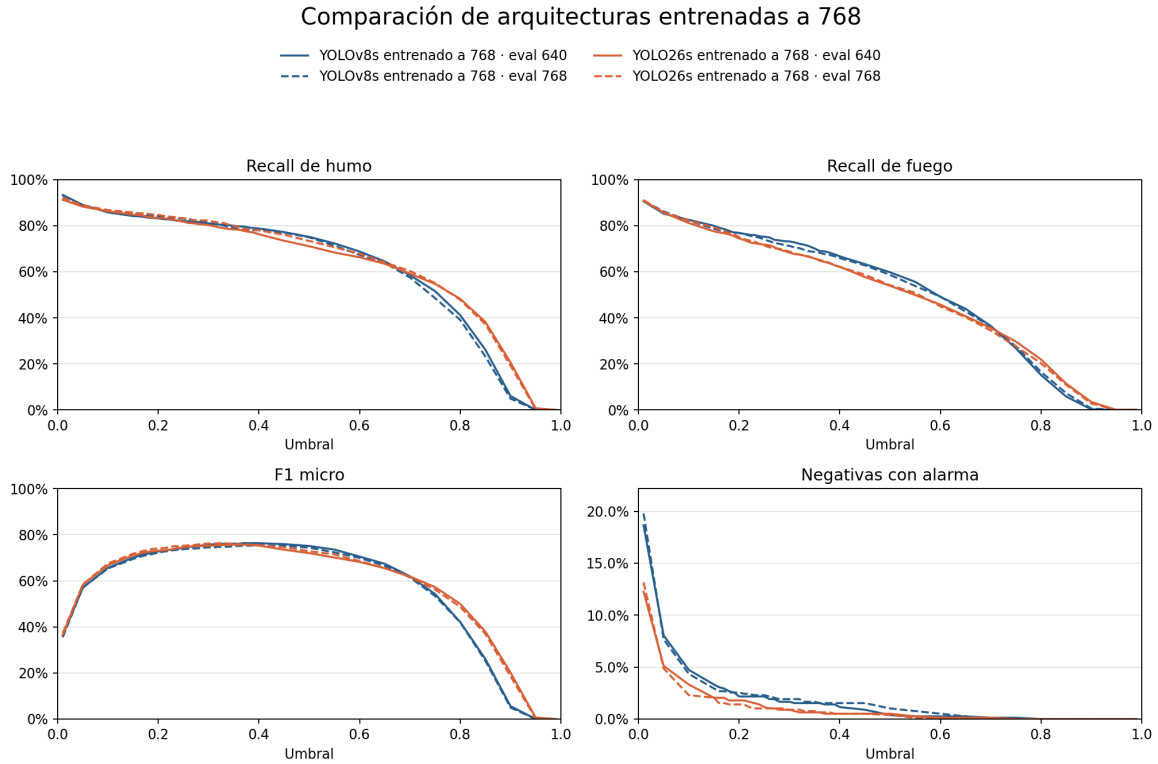


Figura 4: Recall de humo, recall de fuego, F1 micro y proporción de negativas con alarma en función del umbral, para YOLOv8s y YOLO26s entrenados a 768 e inferidos a 640 y a 768.

Esta diferencia puede deberse a la propia arquitectura: YOLO26 es *end-to-end* y no aplica supresión de no máximos (Anexo C), por lo que no se beneficia del mismo modo que YOLOv8s del desajuste entre la resolución de entrenamiento y la de inferencia. En cualquier caso, el resultado experimental llevó a conservar para cada familia la resolución que mejor funcionaba en validación: YOLOv8s a 768 $\rightarrow$ 640 y YOLO26s a 768 $\rightarrow$ 768. Bajo su respectiva mejor configuración, YOLO26s obtiene un recall macro mayor que YOLOv8s (81,97 % frente a 78,38 %) con un F1 micro ligeramente menor y menos alarmas en imágenes negativas, además de conservar un peso menor.

4.4. Ajuste de hiperparámetros de YOLO26s

Una vez fijados el modelo y la resolución de referencia (YOLO26s a 768 píxeles), se exploró si un pequeño ajuste de hiperparámetros podía mejorar todavía más el resultado. Debido al alto coste computacional del entrenamiento, no se pudo realizar un ajuste exhaustivo. En el notebook 08 se puede revisar la estrategia de ajuste: en lugar de entrenar varias configuraciones completas de 100 épocas, se compararon las primeras 50 épocas de cuatro ensayos nuevos frente a las primeras 50 épocas del propio entrenamiento de referencia, recortado a la misma fidelidad para que la comparación fuese justa. Cada ensayo modifica un único bloque interpretable de hiperparámetros, manteniendo fijos el dataset, la resolución, el optimizador, el *batch size* y la semilla. Los cuatro ensayos fueron:

- **HP01:** tasa de aprendizaje inicial de 5×10^{-4} (frente a 1×10^{-3} en la referencia).

- **HP02:** *weight decay* de 1×10^{-3} .
- **HP03:** *data augmentation* más conservador, reduciendo respecto a la referencia la intensidad del jitter de color (HSV), la traslación y el escalado aleatorios (aprox. a la mitad) y la probabilidad de mosaico (del 100 % al 70 %), para evitar ejemplos poco realistas.
- **HP04:** combinación de HP01 y HP03.

Tabla 6: Cribado de hiperparámetros de YOLO26s a 50 épocas, frente al entrenamiento de referencia recortado a la misma fidelidad.

Ensayo	mAP50-95 (época 50)	Diferencia vs. referencia (p.p.)
Referencia (recortada a 50 épocas)	45,78 %	—
HP01 ($lr\ 5 \times 10^{-4}$)	46,12 %	+0,34
HP02 (<i>weight decay</i> 1×10^{-3})	45,32 %	-0,45
HP03 (aumento conservador)	45,23 %	-0,55
HP04 (HP01 + HP03)	45,38 %	-0,40

Solo HP01 mejora ligeramente al recortar la referencia a la misma fidelidad; el resto de ensayos empeora. Los cuatro ensayos se evaluaron también con el barrido completo de umbrales por clase bajo el límite del 1 % de imágenes negativas con alarma (el mismo criterio que se usa en la selección final, Sección 4.5): HP01 vuelve a ser el mejor de los cuatro, con un recall macro del 79,14 % (humo 83,65 %, fuego 74,63 %), seguido de HP04 (78,41 %), HP02 (75,95 %) y HP03 (75,70 %).

La mejora de HP01 es pequeña, se obtuvo con una sola semilla y a una fidelidad reducida (50 de las 100 épocas habituales), por lo que no se considera una diferencia atribuible con garantías al hiperparámetro en sí. Dados estos resultados, el modelo de referencia (YOLO26s a 768, entrenado con los hiperparámetros originales) se mantiene como la configuración que llega a la selección final.

4.5. Selección final en validación

Con la resolución de referencia fijada para cada arquitectura, y descartado el ajuste de hiperparámetros de la sección anterior, quedaban dos candidatos: YOLO26s 768 $\rightarrow$ 768 y YOLOv8s 768 $\rightarrow$ 640. La selección final entre ambos (notebook 09) se realizó exclusivamente sobre validación, endureciendo hasta aproximadamente el 1 % el límite de falsas alarmas del 2 % empleado hasta ahora. Además, se permitió un umbral distinto para humo y fuego. Para cada modelo se exploró una rejilla de pares de umbrales entre 0,16 y 0,60, con paso 0,01. Las configuraciones resultantes se ordenaron por recall macro, usando como criterios de desempate el recall de la clase más débil, el F1 micro y la precisión micro.

Tabla 7: Punto operativo seleccionado en validación, con presupuesto de falsas alarmas del 1 %.

Modelo	Umbral humo/fuego	Recall humo	Recall fuego	Recall macro	Precisión micro	F1 micro
YOLO26s 768 $\rightarrow$ 768	0,36/0,16	79,38 %	78,53 %	78,95 %	68,82 %	73,52 %
YOLOv8s 768 $\rightarrow$ 640	0,49/0,17	75,62 %	78,53 %	77,08 %	68,84 %	72,79 %

Ambos modelos activan 7 de las 783 imágenes negativas (0,89 %) y quedan prácticamente

empatados en precisión micro ($\approx 68,8$ %); YOLO26s se impone por su recall macro y, sobre todo, por reducir en 36 los falsos negativos de humo respecto a YOLOv8s mientras ambos empatan en los 248 falsos negativos de fuego, como recoge la Tabla 8.

Tabla 8: Falsos negativos por clase en el punto operativo seleccionado en validación.

Modelo	FN humo	FN fuego
YOLO26s 768 $\rightarrow$ 768	198	248
YOLOv8s 768 $\rightarrow$ 640	234	248

Por tamaño de objeto (Tabla 9), YOLO26s iguala o supera a YOLOv8s en las tres bandas de humo y en fuego mediano y grande, y solo queda 0,4 puntos por debajo en fuego pequeño.

Tabla 9: Recall por clase y tamaño de objeto en el punto operativo seleccionado en validación. Bandas por área normalizada: pequeño <1 %, mediano 1–10 %, grande ≥ 10 %.

Modelo	Clase	Pequeño	Mediano	Grande
YOLO26s 768 $\rightarrow$ 768	Humo	68,62 %	67,57 %	87,82 %
YOLO26s 768 $\rightarrow$ 768	Fuego	75,92 %	81,79 %	91,55 %
YOLOv8s 768 $\rightarrow$ 640	Humo	63,30 %	63,96 %	84,55 %
YOLOv8s 768 $\rightarrow$ 640	Fuego	76,32 %	81,79 %	87,32 %

Dados los resultados, se selecciona YOLO26s a 768 píxeles, con umbral 0,36 para humo y 0,16 para fuego, como modelo final. Tras congelar el modelo, y ya fuera del proceso de selección, se ejecutó una auditoría de cierre (notebook 10): bajo el mismo criterio del 1 % de imágenes negativas con alarma, se recalcularon las 14 combinaciones de resolución de entrenamiento e inferencia (a 640 y 768 píxeles) de los cuatro modelos base (YOLOv8n, YOLOv8s, YOLO26n, YOLO26s) estudiadas en las secciones anteriores, añadiendo también los cuatro ensayos de ajuste de hiperparámetros de la Sección 4.4 a modo de referencia.

Esta auditoría (Figura 5) muestra que el mayor recall no corresponde a la configuración ya elegida, sino a YOLO26s 640 $\rightarrow$ 640 (79,86 % frente al 78,95 % de YOLO26s 768 $\rightarrow$ 768). Sin embargo, ese recall está mucho peor repartido entre clases (humo 84,48 % frente a fuego 75,24 %, 9,2 puntos de diferencia) que el de la configuración finalista (humo 79,38 % frente a fuego 78,53 %, menos de 1 punto), y ofrece 3,3 puntos menos de recall en fuego, la clase más difícil y más relevante desde el punto de vista operativo. Por ello no se reabrió la selección tras ampliar la auditoría a todas las configuraciones bajo el criterio ya fijado del 1 %: se mantuvo YOLO26s 768 $\rightarrow$ 768, con menor recall macro que YOLO26s 640 $\rightarrow$ 640, pero mejor equilibrio entre clases y mayor recall de fuego.

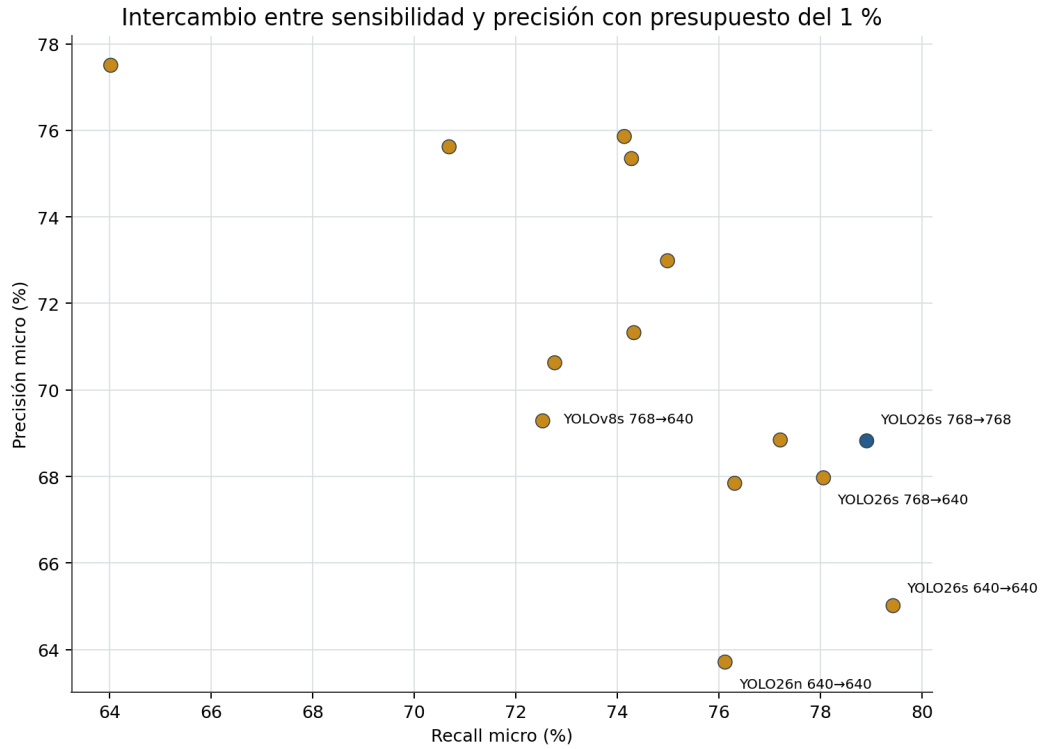


Figura 5: Recall micro vs precisión micro de todas las configuraciones bajo el límite del 1 % de alarmas en negativas. YOLO26s 768 → 768 (punto azul) fue la configuración finalista; YOLO26s 640 → 640 obtiene mayor recall pero con menor precisión y peor equilibrio entre clases.

5. Evaluación y resultados

El modelo elegido finalmente es YOLO26s entrenado e inferido a 768 píxeles, con umbrales de decisión específicos por clase: 0,36 para humo y 0,16 para fuego (Sección 4.5). Estos umbrales, y solo estos, se evaluaron una única vez sobre el conjunto de test (notebook 11), sin reajustar pesos, resolución ni umbrales a partir del resultado. A continuación se presentan las métricas finales sobre test de dicha evaluación.

5.1. Métricas en test

Tabla 10: Métricas con los umbrales de despliegue (humo 0,36, fuego 0,16) y métricas estándar, sobre las 4.306 imágenes de test.

Métrica	Global	Humo	Fuego
Precisión	67,88 %	84,41 %	58,26 %
Recall	77,32 %	79,45 %	75,61 %
F1	72,29 %	81,85 %	65,81 %
mAP50	79,27 %	85,67 %	72,86 %
mAP50-95	46,38 %	53,65 %	39,10 %

Precisión, recall y F1 se calculan con los umbrales de confianza fijos calculados en el apartado anterior –0,36 para humo, 0,16 para fuego–. La columna Global es micro: se suman los aciertos y errores de humo y fuego antes de calcular la métrica; las columnas Humo y Fuego son la precisión, el recall y el F1 de cada clase por separado.

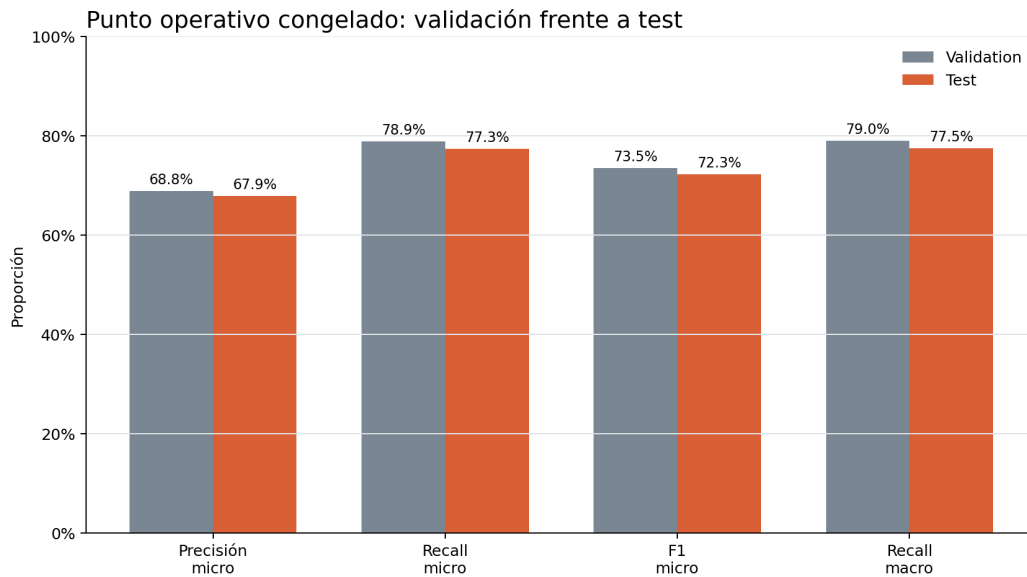


Figura 6: Precisión, recall y F1 micro, y recall macro, con los umbrales de despliegue congelados, en validación frente a test.

Frente a validación, la precisión micro cae 0,94 puntos, el recall micro 1,60 y el F1 micro 1,23 (Figura 6). La caída más pronunciada es la del recall de la clase peor –fuego–, con 2,92 puntos menos (de 78,53 % a 75,61 %); el recall macro baja 1,42 puntos, hasta 77,53 %. La proporción de imágenes negativas con alarma sube del 0,89 % en validación al 1,05 % en test (21 de 2.005 imágenes), superando ligeramente el límite operativo del 1 % fijado previamente. En conjunto, el modelo generaliza bien –las caídas son pequeñas y esperables entre dos particiones del mismo dataset–, aunque persiste el mismo patrón ya visto en validación: fuego es la clase que peor generaliza.

5.2. Recall por tamaño de objeto

Tabla 11: Recall por clase y tamaño de objeto en test, con la diferencia frente a validación. Bandas por área normalizada: pequeño <1 %, mediano 1–10 %, grande ≥ 10 %.

Clase	Tamaño	Recall test	Δ vs. validación (p.p.)
Humo	Pequeño	66,06 %	-2,56
Humo	Mediano	69,32 %	+1,75
Humo	Grande	88,94 %	+1,12
Fuego	Pequeño	73,60 %	-2,32
Fuego	Mediano	79,25 %	-2,54
Fuego	Grande	79,41 %	-12,14

Como se puede observar en la tabla 11, el patrón del recall por tamaño de objeto es similar al de validación, aumentando con el tamaño en ambas clases. La caída más llamativa es la de fuego grande, 12,14 puntos por debajo de validación (91,55 % a 79,41 %). La caída puede deberse en parte al tamaño reducido de esta banda en ambas particiones (71 cajas en validación, 170 en test), que hace que el recall sea más sensible a errores individuales.

5.3. Análisis de errores

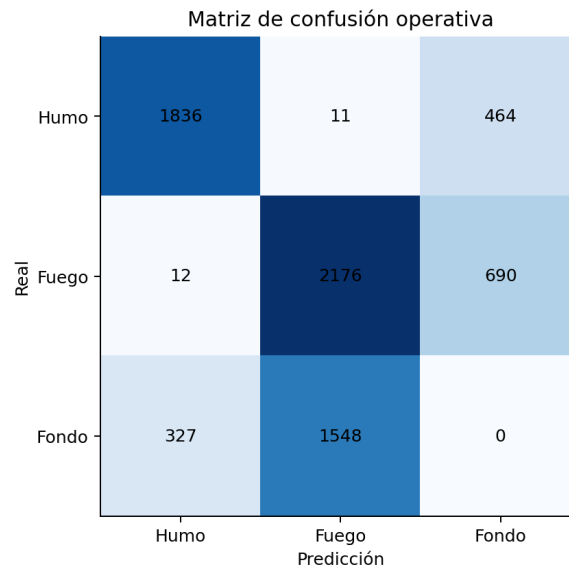


Figura 7: Matriz de confusión en test, con los umbrales de despliegue (humo 0,36, fuego 0,16).

La matriz de confusión muestra que casi todos los errores son omisiones o alarmas sobre fondo, no confusión entre clases: solo 11 cajas de humo se predicen como fuego y 12 de fuego como humo. Fuego genera muchos más falsos positivos sobre fondo que humo (1.548 frente a 327), el mismo patrón asimétrico ya observado en validación. De las 2.005 imágenes negativas, 14 (0,70 %) reciben una falsa alarma de humo y 7 (0,35 %) de fuego, sin solapamiento entre ambas. Tras una revisión manual, se confirma que los falsos positivos se concentran en escenas con nubes, bruma, reflejos o iluminación intensa y los falsos negativos, en señales pequeñas, ocluidas o de bajo contraste. También se observa cómo muchos errores son debidos a que la predicción no coincide exactamente con la caja real, aunque sí se superpone parcialmente. Esto es relevante porque el modelo puede detectar la señal de incendio, pero no ajusta bien la caja probablemente debido a la variabilidad de formas y texturas de humo y fuego.

5.4. Ejemplos de inferencia

La Figura 8 recoge seis casos dirigidos –no una muestra aleatoria– que ilustran los patrones anteriores: alarmas sobre negativas, confusión entre clases, un falso negativo y aciertos representativos. En este caso se puede observar en el falso negativo de fuego cómo el modelo detecta la señal, pero no ajusta bien la caja.

02 alarma_negativa · WEB10398.jpg



04 confusion_clase · WEB11160.jpg



03 alarma_negativa · WEB10418.jpg



07 falso_negativo · WEB10605.jpg



11 acierto_representativo · AoF07833.jpg



12 acierto_representativo · WEB10741.jpg



Figura 8: Seis ejemplos dirigidos de inferencia en test: alarmas sobre imágenes negativas, confusión entre clases, falso negativo y aciertos representativos. Azul = predicción correcta (TP); naranja = predicción incorrecta (FP); verde = caja real detectada; amarillo = caja real omitida (FN).

6. Interpretabilidad

El análisis de interpretabilidad se aplicó al modelo final: YOLO26s entrenado e inferido a 768 píxeles, con umbral 0,36 para humo y 0,16 para fuego sobre el conjunto de validación. El objetivo es comprobar de forma cualitativa qué zonas de la imagen sostienen algunas predicciones representativas. El análisis se puede ver en el notebook 12.

Se seleccionaron seis casos: un acierto, una falsa alarma en imagen negativa y una omisión limítrofe (un falso negativo cercano a ser detectado correctamente) para cada una de las dos clases. Sobre ellos se combinaron tres herramientas complementarias. En primer lugar, Eigen-CAM multiescala resume las activaciones internas del detector y muestra qué zonas concentran mayor respuesta visual. En estos mapas, las zonas cálidas (amarillo, naranja o rojo claro) indican mayor activación, mientras que las zonas moradas u oscuras corresponden a menor activación. En segundo lugar, la sensibilidad por oclusión oculta regiones de la imagen y mide cuánto cambia la confianza de la predicción. Por último, las curvas de eliminación comparan la caída de confianza al retirar las regiones más relevantes frente a retirar regiones aleatorias.

Caso 1

Acierto de humo · WEB06295.jpg · confianza 0.893



Capas: L16: 448×800, L19: 448×800, L22: 448×800. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Caso 2

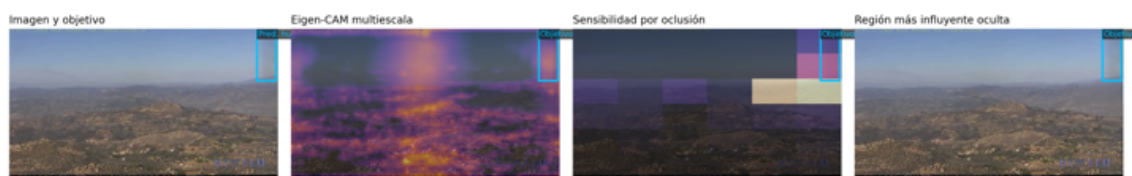
Acierto de fuego · WEB03832.jpg · confianza 0.841



Capas: L16: 720×1280, L19: 720×1280, L22: 720×1280. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Caso 3

Falsa alarma de humo · WEB00156.jpg · confianza 0.646

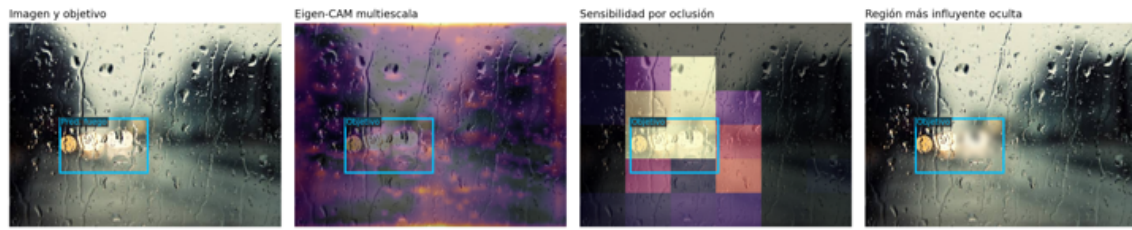


Capas: L16: 360×640, L19: 360×640, L22: 360×640. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Figura 9: Resultados cualitativos de interpretabilidad para los casos 1–3. Para cada caso se muestra la imagen original con la predicción, el mapa Eigen-CAM, la sensibilidad por oclusión y la región cuya ocultación produce mayor caída de confianza. En Eigen-CAM, los colores cálidos indican mayor activación del detector.

Caso 4

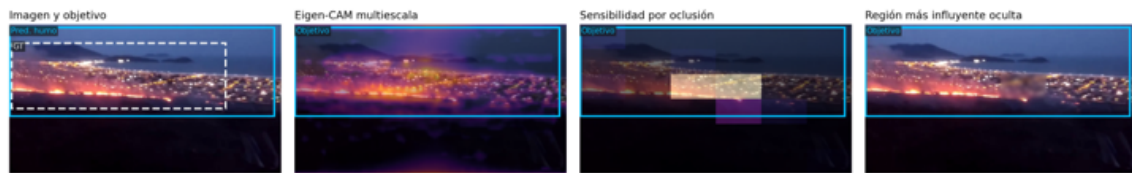
Falsa alarma de fuego · WEB02251.jpg · confianza 0.518



Capas: L16: 480×640, L19: 480×640, L22: 480×640. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Caso 5

Omisión límite de humo · WEB05832.jpg · confianza 0.357



Capas: L16: 352×640, L19: 352×640, L22: 352×640. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Caso 6

Omisión límite de fuego · WEB05014.jpg · confianza 0.159



Capas: L16: 360×640, L19: 360×640, L22: 360×640. Azul: predicción objetivo; GT discontinua. Eigen-CAM es activación no específica de clase; oclusión mide caída relativa de la confianza.

Figura 10: Resultados cualitativos de interpretabilidad para los casos 4–6. Para cada caso se muestra la imagen original con la predicción analizada, el mapa Eigen-CAM multiescala, la sensibilidad por oclusión y la región cuya ocultación produce mayor caída de confianza. En Eigen-CAM, los colores cálidos indican mayor activación del detector.

Los tres métodos aportan lecturas complementarias. Eigen-CAM ofrece una primera aproximación visual a las zonas con mayor activación interna del detector. En los aciertos, las activaciones tienden a concentrarse cerca de las regiones de humo o fuego, aunque no siempre quedan restringidas al interior de la caja anotada. En las falsas alarmas, en cambio, los mapas muestran activaciones sobre patrones visualmente parecidos a humo o fuego, como nubes, bruma, reflejos o zonas de iluminación intensa. Esto ayuda a entender por qué esas escenas pueden resultar ambiguas para el modelo.

La sensibilidad por oclusión permite comprobar de forma más directa qué zonas afectan a la confianza: se ocultan regiones de la imagen y se mide cuánto cambia la puntuación de la predicción. En los seis casos analizados, las regiones cuya ocultación produce mayor caída de confianza suelen coincidir con la señal detectada o con contexto visual muy cercano. Este resultado es importante porque muestra que el modelo no depende únicamente de la caja anotada, sino también de información contextual: textura, fondo, bordes difusos,

iluminación y patrones espaciales asociados al humo o al fuego.

Por último, la prueba de eliminación resume cuantitativamente esa misma idea. Al retirar el 25 % de las regiones más relevantes, el modelo conservó solo una mediana del 3,8 % de su confianza inicial; al retirar regiones aleatorias conservó el 75,6 %. Dicho de otro modo, la confianza cayó un 96,2 % frente a un 24,4 %, respectivamente. La caída fue mayor con la eliminación dirigida en los seis casos, lo que indica que los mapas capturan información útil para la predicción.

7. Despliegue y productivización

Además del entrenamiento y la evaluación del modelo, el proyecto incluye un despliegue real en un Raspberry Pi 5, con sus propias métricas de rendimiento medidas sobre el dispositivo, y una aplicación operativa, Vigía.

7.1. Raspberry Pi 5

El modelo PyTorch final se probó primero tal cual, como referencia: sobre seis imágenes y 20 repeticiones a 768 píxeles, la latencia media en Raspberry Pi 5 fue de 2.621,36 ms (0,381 FPS), un rendimiento insuficiente para uso continuo. Se exportaron entonces dos variantes a formato NCNN, a 768 y 640 píxeles. NCNN es un framework de inferencia en C++ optimizado para CPUs ARM que, mediante instrucciones vectoriales y cuantización de los pesos, reduce drásticamente la latencia de un modelo ya entrenado a cambio de un comportamiento numérico ligeramente distinto al original. Aplicar directamente los umbrales congelados del modelo de PyTorch (0,36/0,16) a la variante NCNN a 768 elevaba las falsas alarmas en validación de 7 a 10 de 783 imágenes negativas (1,28 %), por encima del límite del 1 % fijado en la Sección 4.5. Por ello ambas variantes se recalibraron por separado, quedando en 0,415/0,165 para 768 px y 0,365/0,165 para 640 px; con esos umbrales, las dos vuelven a situarse en 0,89 % de falsas alarmas.

Tabla 12: Rendimiento medido en Raspberry Pi 5 (seis imágenes, 20 repeticiones).

Backend	Resolución	Latencia	FPS
PyTorch	768	2.621,36 ms	0,381
NCNN	768	252,04 ms	3,968
NCNN	640	167,41 ms	5,973

Frente a NCNN a 768 px, la variante a 640 px reduce la latencia un 33,58 % y aumenta el rendimiento un 50,55 %, con ambos modelos dentro del límite de falsas alarmas. El coste en métricas es pequeño y mixto: el recall de fuego baja 1,04 puntos y el F1 micro 0,78, pero el recall macro sube 0,47 puntos porque el de humo mejora casi 2 puntos; ninguna de las dos resoluciones domina a la otra en todas las métricas. Dado que la mejora de velocidad es sustancial y el coste queda compensado en parte por el recall macro, la recomendación resultante de la comparación es NCNN a 640 px para el despliegue en el dispositivo.

Tabla 13: Métricas de validación de las variantes NCNN en su punto de calibración (humo 0,415/fuego 0,165 a 768 px; humo 0,365/fuego 0,165 a 640 px).

Clase	Métrica	NCNN 768	NCNN 640
Global	Precisión (micro)	69,01 %	67,40 %
Global	Recall (micro)	79,05 %	79,39 %
Global	Recall (macro)	78,90 %	79,37 %
Global	F1 (micro)	73,69 %	72,90 %
Global	Falsas alarmas (validación)	0,89 %	0,89 %
Humo	Precisión	87,69 %	85,68 %
Humo	Recall	77,19 %	79,17 %
Humo	F1	82,11 %	82,30 %
Fuego	Precisión	59,00 %	57,29 %
Fuego	Recall	80,61 %	79,57 %
Fuego	F1	68,13 %	66,62 %

Además, en los tres backends la temperatura se mantuvo entre 45,75 y 50,15 °C y el consumo máximo de memoria por debajo de 600 MB.

7.2. Aplicación de detección

Vigía (`fire_app` en el repositorio) es la aplicación operativa del TFM: una interfaz web que permite analizar una imagen o un vídeo, o usar la cámara del navegador, con el modelo final congelado –YOLO26s a 768 píxeles– o con la variante NCNN preparada para la Raspberry Pi. Está construida sobre FastAPI, un framework de Python para crear APIs web, que expone tanto peticiones HTTP normales como una conexión en tiempo real (WebSocket) para el modo cámara; el propio servidor ejecuta la inferencia con Ultralytics YOLO y OpenCV, y sirve una interfaz sencilla de HTML, CSS y JavaScript sin frameworks adicionales.

Para que una alarma no se dispare por un único fotograma suelto, una detección solo se confirma como incidente si la clase se mantiene visible de forma continuada durante un tiempo mínimo; una interrupción breve no reinicia esa confirmación, pero una más larga sí la reinicia, y una vez cerrado un incidente hay un margen antes de poder generar uno nuevo, para no repetir avisos del mismo evento. Esta regla se aplica por separado para humo y fuego, y cada evento queda registrado para poder revisarlo después.

Cuando se confirma un incidente, Vigía puede avisar automáticamente mediante un bot de Telegram, enviando la clase detectada, la confianza, el momento y una imagen anotada; si un mismo aviso incluye humo y fuego a la vez, se prioriza fuego.

8. Conclusiones y trabajo futuro

El trabajo desarrollado demuestra que es posible construir una cadena completa para la detección temprana de humo y fuego mediante visión artificial, desde la preparación del conjunto de datos hasta una aplicación desplegable. No se ha evaluado únicamente un detector aislado: también se han definido umbrales operativos por clase, se ha separado la selección en validación de la evaluación final en test, se ha estudiado el comportamiento de los errores, se han aplicado técnicas de interpretabilidad y se ha medido el rendimiento en un dispositivo de recursos limitados.

La decisión central del proyecto ha sido priorizar el recall bajo un límite explícito de falsas alarmas, ya que en un sistema de aviso temprano un falso negativo tiene un coste potencialmente mayor que una alarma revisable. El modelo final seleccionado fue YOLO26s entrenado e inferido a 768 píxeles, con umbrales 0,36 para humo y 0,16 para fuego. En test alcanzó un recall del 79,45 % en humo y del 75,61 % en fuego, con un F1 global del 72,29 %. La proporción de imágenes negativas con alarma fue del 1,05 %, ligeramente por encima del 1 % fijado en validación, resultado que se mantiene sin reajuste para reflejar de forma honesta la generalización.

El análisis de errores muestra que los falsos positivos aparecen sobre todo en escenas con nubes, bruma, reflejos o iluminación intensa, mientras que los falsos negativos se concentran en señales pequeñas, ocluidas o de bajo contraste. La interpretabilidad mediante Eigen-CAM, oclusión y eliminación confirma que el modelo utiliza regiones visualmente relevantes, aunque también depende del contexto cercano, la textura y la iluminación, algo esperable en fenómenos con bordes difusos como humo y fuego.

El despliegue confirma la viabilidad técnica del sistema en hardware de bajo coste siempre que se optimice el backend de inferencia. La aplicación Vigía completa el prototipo al permitir análisis de imágenes, vídeo y cámara en directo, confirmación temporal de alarmas, registro de eventos y aviso por Telegram. En conjunto, el resultado es un prototipo avanzado y auditable: antes de un despliegue real sería necesario validarlo con vídeo continuo, cámaras reales, una prueba de carga prolongada y condiciones ambientales variadas.

Como trabajo futuro, las prioridades se agrupan en tres frentes. En datos y modelo: ampliar el dataset con humo incipiente, fuego pequeño, escenas nocturnas y negativos difíciles, e incorporar seguimiento temporal entre fotogramas. En validación de campo: probar el sistema con vídeo continuo en cámaras de vigilancia o drones, midiendo en ese contexto real falsas alarmas por hora, tiempo hasta detección, estabilidad del hardware y coste de mantenimiento. En producto y despliegue: escalar Vigía a varias cámaras simultáneas, explorar aceleradores adicionales más allá de la cuantización NCNN, y aprovechar el propio registro de eventos de Vigía en campo como base para futuros reentrenamientos.

Desde una perspectiva de negocio, Vigía puede plantearse como un producto de monitorización preventiva para administraciones públicas, servicios de emergencia, gestores forestales, infraestructuras críticas y empresas con activos expuestos a riesgo de incendio. En un contexto de incendios forestales cada vez más frecuentes y extensos, una herramienta así no sustituye a los servicios de extinción ni a la supervisión humana, pero puede reducir el tiempo entre los primeros indicios visuales de humo o fuego y una alerta revisable, aportando minutos valiosos de anticipación.

9. Bibliografía y recursos

Referencias

- [1] Ministerio para la Transición Ecológica y el Reto Demográfico. *Los incendios forestales en España. 1 de enero – 31 de diciembre 2025. Avance informativo*. Área de Defensa contra Incendios Forestales, mayo de 2026. https://www.miteco.gob.es/content/dam/miteco/es/biodiversidad/temas/incendios-forestales/Avance_IIFF_2025_.pdf
- [2] Ministerio para la Transición Ecológica y el Reto Demográfico. *Avance informativo de incendios forestales del 1 de enero al 30 de agosto de 2026*. Datos provisionales proporcionados por las comunidades autónomas. <https://www.miteco.gob.es/content/dam/miteco/es/biodiversidad/temas/incendios-forestales/Avance%20informativo%2030%20de%20agosto%20de%202026.pdf>
- [3] Gaia Solutions on Demand. *D-Fire Dataset: an image dataset for fire and smoke detection*. Repositorio, documentación y licencia CC0 1.0 del conjunto de datos. <https://github.com/gaia-solutions-on-demand/DFireDataset>
- [4] P. V. A. B. de Venâncio, A. C. Lisboa y A. V. Barbosa. *An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices*. Neural Computing and Applications, 2022. <https://link.springer.com/article/10.1007/s00521-022-07467-z>
- [5] S. Ren, K. He, R. Girshick y J. Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. Advances in Neural Information Processing Systems, 2015. <https://proceedings.neurips.cc/paper/2015/hash/14bfa6bb14875e45bba028a21ed38046-Abstract.html>
- [6] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu y A. C. Berg. *SSD: Single Shot MultiBox Detector*. ECCV, 2016. <https://arxiv.org/abs/1512.02325>
- [7] J. Redmon, S. Divvala, R. Girshick y A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. CVPR, 2016. <https://arxiv.org/abs/1506.02640>
- [8] Ultralytics. *YOLO Architecture Explained: From YOLOv3 to YOLO26*. <https://docs.ultralytics.com/guides/yolo-architecture/>
- [9] Ultralytics. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. Documentación del modelo YOLO26. <https://docs.ultralytics.com/models/yolo26/>
- [10] R. Sapkota y M. Karkee. *Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8, and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition*. arXiv:2510.09653v3, 2026. <https://arxiv.org/html/2510.09653v3>

A. Acceso al repositorio del proyecto

El código, los notebooks reproducibles, la configuración de entrenamiento y los scripts auxiliares del proyecto están disponibles en un repositorio Git accesible a partir del siguiente enlace:

https://github.com/Elitron22/vigia_fire_detection

Para obtener una copia local del proyecto se puede ejecutar:

```
1 git clone https://github.com/Elitron22/vigia_fire_detection.git
2 cd vigia_fire_detection
```

O bien por SSH:

```
1 git clone git@github.com:Elitron22/vigia_fire_detection.git
2 cd vigia_fire_detection
```

Dentro del repositorio, los notebooks se encuentran en la carpeta **notebooks/**. Los más relevantes para reproducir la memoria son:

- **notebooks/01_DFire_preparacion_dataset.ipynb**: preparación del dataset.
- **notebooks/02_DFire_entrenamiento_modelos.ipynb**: entrenamientos.
- **notebooks/03_DFire_evaluacion_modelos.ipynb**: evaluación estándar en validación.
- **notebooks/06_DFire_comparacion_resolucion.ipynb**: estudio de resolución.
- **notebooks/09_DFire_seleccion_final_validacion.ipynb**: selección del modelo final.
- **notebooks/11_DFire_evaluacion_final_test.ipynb**: evaluación final sobre test.

La puesta en marcha del contenedor Docker y el uso de la aplicación web —incluyendo requisitos previos, variables de entorno y comandos de arranque— se detallan paso a paso en el fichero **README.md** del repositorio.

El repositorio no incluye el dataset D-Fire original ni la versión revisada del mismo que se menciona en la memoria debido al alto peso de las imágenes. Para reproducir el entrenamiento desde cero revisar el **README.md** del repositorio.

Las credenciales externas no forman parte de la entrega: tokens y claves de servicios como Telegram deberán definirse mediante variables de entorno.

B. Descripción de las métricas de evaluación

Este anexo define con detalle cada una de las métricas empleadas a lo largo de la memoria para comparar y seleccionar modelos.

B.1. Confianza, IoU y emparejamiento de cajas

Cada predicción del detector incluye una **confianza**: un valor entre 0 y 1 que expresa cuánto “cree” el modelo que la caja propuesta contiene realmente un objeto de la clase indicada. El **umbral de confianza** descarta las predicciones por debajo de ese valor antes de calcular el resto de métricas; un umbral bajo favorece el recall (se detectan más casos reales) a costa de más falsos positivos, y un umbral alto favorece la precisión a costa de perder detecciones.

El **IoU** (*Intersection over Union*, intersección sobre unión) mide el solapamiento entre dos cajas delimitadoras como el área de su intersección dividida entre el área de su unión. Toma valores entre 0 (sin solapamiento) y 1 (cajas idénticas). El **IoU de emparejamiento** es el umbral de IoU a partir del cual una predicción se considera acierto sobre una caja real: por ejemplo, con IoU de emparejamiento 0,50, una predicción de la clase correcta que solape al menos un 50 % con la caja real cuenta como verdadero positivo.

El **IoU de NMS** (*Non-Maximum Suppression*, supresión de no máximos) es distinto: se aplica antes de comparar con la anotación real y sirve para eliminar predicciones redundantes sobre el mismo objeto, conservando solo la de mayor confianza entre aquellas que se solapan por encima de dicho umbral. YOLOv8 aplica NMS con IoU 0,70; YOLO26 es *end-to-end* y no lo necesita, ya que su cabeza de detección produce directamente una única predicción por objeto (Anexo C).

B.2. Verdaderos positivos, falsos positivos y falsos negativos

Fijados un umbral de confianza y un IoU de emparejamiento, cada predicción y cada anotación real se clasifican como:

- **Verdadero positivo (TP)**: predicción de la clase correcta que empareja con una caja real no asignada previamente.
- **Falso positivo (FP)**: predicción que no empareja con ninguna caja real, ya sea porque no hay objeto en esa región o porque la clase es incorrecta.
- **Falso negativo (FN)**: caja real que ningún acierto ha cubierto, es decir, un objeto que el modelo no detectó.

B.3. Precisión, recall y F1

A partir de TP, FP y FN se definen:

$$\text{Precisión} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

Estas métricas pueden agregarse de dos formas cuando hay varias clases (humo y fuego):

- **Micro:** se suman los TP, FP y FN de todas las clases antes de calcular precisión, recall y F1. Da más peso a la clase con más casos y refleja el comportamiento agregado del sistema tal como lo vería un usuario, sin distinguir el tipo de alerta.
- **Macro:** se calcula la métrica por separado para cada clase y después se promedia. Da el mismo peso a humo y a fuego independientemente de cuántos ejemplos tenga cada uno, por lo que penaliza más si una clase concreta (por ejemplo, fuego) funciona mal aunque la otra compense el promedio agregado.

B.4. AP, mAP50 y mAP50-95

La **AP** (*Average Precision*, precisión media) de una clase resume su curva de precisión-recall en un único número: se calcula variando el umbral de confianza de 1 a 0, registrando en cada punto la pareja (precisión, recall), y promediando la precisión a lo largo de todo el rango de recall. A diferencia de la precisión y el recall a un único umbral de confianza, la AP no depende de ningún umbral fijo: describe la calidad global del ranking de predicciones del modelo.

El **mAP** (*mean Average Precision*) promedia la AP de todas las clases. La memoria usa dos variantes, ambas calculadas por el barrido estándar de Ultralytics:

- **mAP50:** la AP se calcula con un único IoU de emparejamiento de 0,50. Es más permisivo con la localización exacta de la caja y más fácil de interpretar.
- **mAP50-95:** la AP se calcula repetidamente para IoU de emparejamiento entre 0,50 y 0,95 en pasos de 0,05, y se promedian los diez valores resultantes. Es más exigente porque penaliza cajas que detectan el objeto pero lo delimitan de forma imprecisa, y es la métrica estándar de referencia en detección de objetos.

B.5. Métricas operativas propias del proyecto

Las métricas anteriores son estándar en detección de objetos, pero no capturan directamente el riesgo de un sistema de aviso temprano: una falsa alarma frecuente sobre escenas sin riesgo real erosiona la confianza en el sistema. Por ello se añadieron métricas operativas propias:

- **Imágenes negativas con alarma**, el número de imágenes sin ninguna anotación real (ni humo ni fuego) para las que el modelo emite al menos una predicción por encima del umbral de confianza. Se expresa como fracción del total de imágenes negativas de la partición (por ejemplo, 7/783) y es, en la práctica, la tasa de falsos positivos a nivel de imagen en el escenario más favorable para el sistema, cuando no hay ningún riesgo presente.

C. Arquitectura de YOLO26

Este anexo amplía la descripción de la arquitectura de YOLO26. La idea principal es que el modelo no analiza la imagen de una sola forma, sino que la transforma progresivamente en representaciones internas cada vez más útiles para localizar objetos. En una arquitectura YOLO moderna se distinguen tres bloques: *backbone*, *neck* y *head*. La Figura 11 resume esta organización.

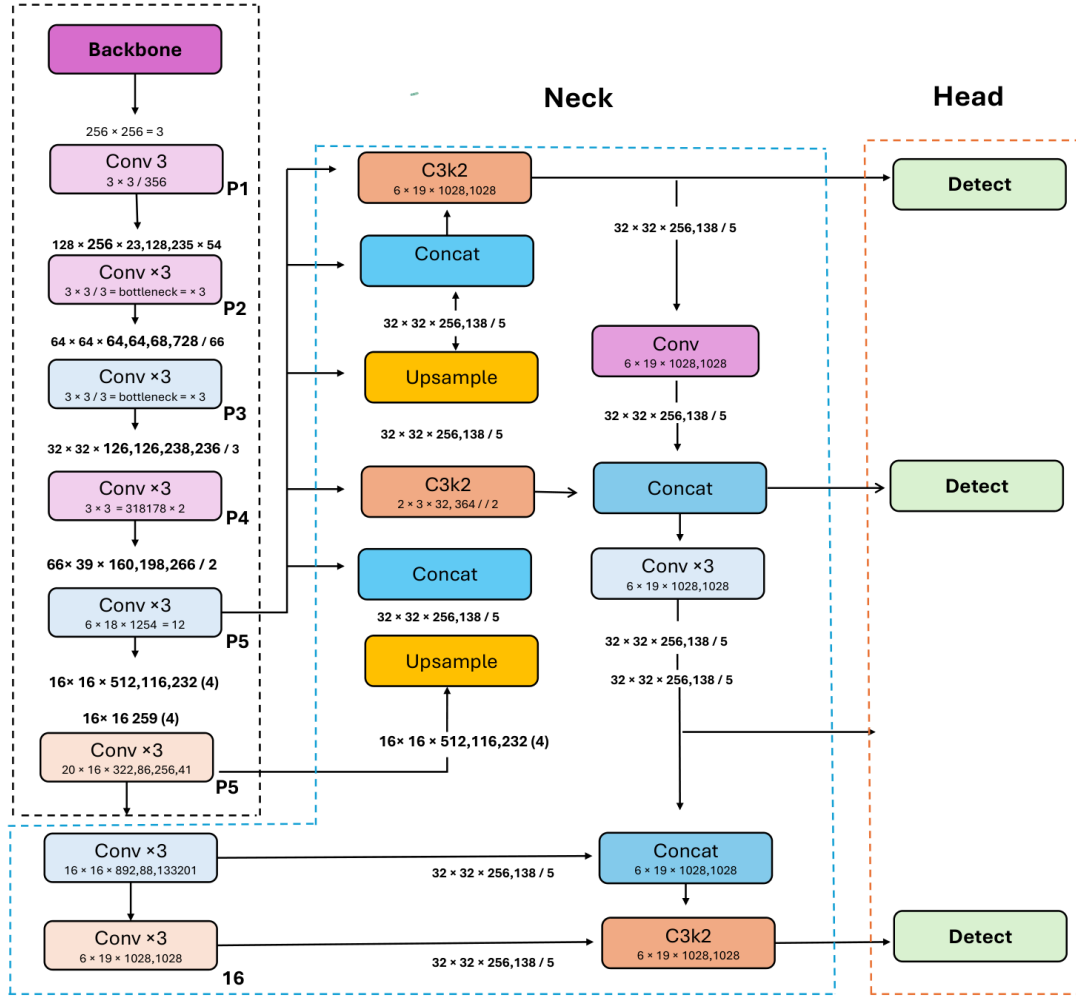


Figura 11: Arquitectura central de YOLO26. Fuente: Sapkota y Karkee [10].

C.1. Entrada y extracción de características

La entrada del modelo es una imagen redimensionada a una resolución fija. A partir de ella, el *backbone* actúa como extractor de características. En las primeras capas se conservan detalles locales, como bordes, cambios de color o texturas. Conforme la información avanza por la red, la resolución espacial disminuye y las características se vuelven más abstractas: ya no representan píxeles concretos, sino patrones visuales más complejos.

En la figura, esta parte corresponde al bloque situado a la izquierda. Las salidas intermedias aparecen marcadas como P1, P2, P3, P4 y P5. Estas salidas no son predicciones finales; son mapas de características con distinta resolución. Los mapas de mayor resolución conservan

mejor la posición de elementos pequeños, mientras que los mapas más profundos contienen más contexto semántico.

C.2. Fusión multiescala

El *neck* recibe mapas de características procedentes de distintas profundidades del *backbone* y los combina. Esta etapa es necesaria porque los objetos no aparecen siempre al mismo tamaño.

Para combinar información de varias escalas, la arquitectura utiliza operaciones como *upsample*, *concat* y bloques internos como C3k2. La operación *upsample* aumenta la resolución de un mapa de características para poder alinearlos con otro de mayor tamaño. La operación *concat* une canales de información procedentes de diferentes niveles, mientras que los bloques C3k2 procesan y enriquecen esos mapas fusionados mediante operaciones convolucionales. De esta forma, el modelo combina detalle espacial y contexto semántico sin aumentar excesivamente el coste computacional. En términos prácticos, el *neck* ayuda a que el detector no dependa de una única escala.

C.3. Cabeza de detección

La *head* es la parte final del detector. Su función es transformar los mapas de características fusionados que salen del *neck* en predicciones interpretables: cajas delimitadoras, clases y puntuaciones de confianza. En este proyecto, las clases de interés son humo y fuego, por lo que cada predicción indica una región de la imagen, una clase asignada de las dos y una confianza asociada.

En versiones anteriores de muchos detectores era habitual partir de un conjunto de cajas predefinidas, llamadas *anchors*, que servían como plantillas iniciales para proponer posibles objetos. Las versiones modernas de YOLO reducen esta dependencia: el modelo aprende a predecir directamente la posición de los objetos a partir de los mapas de características, sin necesitar tantas cajas iniciales fijadas de antemano [8].

Además, YOLO26 está diseñado para simplificar el proceso completo de detección. En lugar de generar muchas cajas candidatas y aplicar después un filtrado adicional para eliminar duplicados, la arquitectura busca producir predicciones finales de forma más directa. Esto puede reducir la latencia y facilitar el despliegue en sistemas donde importa que la respuesta sea rápida, como una cámara en directo o un dispositivo de bajo coste [9].

C.4. Escalas de modelo y despliegue

Otro aspecto relevante de YOLO es que suele ofrecer varias escalas de modelo. Las variantes pequeñas priorizan velocidad y bajo consumo de recursos, mientras que las variantes mayores aumentan capacidad a costa de más memoria y tiempo de inferencia. Esta familia de tamaños permite escoger el modelo según el entorno final.

En este trabajo, este punto es especialmente importante porque el sistema no solo debe funcionar en un ordenador de desarrollo, sino también en un dispositivo de bajos recursos.